

シェルソート

「挿入ソートの弱点を克服した、進化版の挿入ソート」です。

分割統治の考え方を取り入れ

- 1. グループ分け(分割)する
- 2. グループごとの整列を行う
- 3. 統合する

を繰り返すことで、挿入ソートを高速化します。

なお、その名前は、考案者のドナルド・シェル氏の名前に由来します。

1. なぜシェルソートが必要なの？（背景）

挿入ソートには、「遠く離れた場所にあるべきデータでも、隣同士を1つずつ比較して挿入場所を探さなければならぬ」という弱点がありました。

例えば、一番左端に最大値があった場合、右端に挿入するまで、何度も比較を繰り返す必要があり、効率が悪かったのです。シェルソートは、「**最初はざっくり大股で動かし、最後に細かく整える**」ことで、この問題を解決しました。

2. アルゴリズムの仕組み

シェルソートは、以下の3ステップで行います。

- 1. **グループ分け**：データを一定の間隔（ギャップ）で飛び飛びに選び、いくつかのグループに分けます。
- 2. **グループ内整列**：分けたグループごとに「挿入ソート」を行います。
- 3. **統合**：整列を終えたグループを統合します。
- 4. **間隔を詰める**：間隔をどんどん狭めながら、グループ分けと統合を繰り返し、最後は間隔を1（普通の挿入ソート）にして仕上げます。

3. 具体的な例

例えば [8, 3, 1, 2, 7, 5, 6, 4] というデータを考えてみましょう。今回は、「元の位置に戻す」というステップがあるので、下のように図示します。

元の位置	値	図示
1	8	#####
2	3	###
3	1	#
4	2	##

元の位置	値	図示
5	7	#####
6	5	#####
7	6	#####
8	4	####

【ステップ1：間隔「4」で整列】

1. 4つ飛ばしのペア（8と7、3と5、1と6、2と4）を作成します。

グループ1

元の位置	値	図示
1	8	#####
5	7	#####

グループ2

元の位置	値	図示
2	3	###
6	5	#####

グループ3

元の位置	値	図示
3	1	#
7	6	#####

グループ4

元の位置	値	図示
4	2	##
8	4	####

2. グループ内で比較・整列します。

グループ1 : $8 > 7$ なので、位置1と位置5を入れ替え(8を位置5に、7を位置1に)

元の位置	値	図示
1	7	#####
5	8	#####

グループ2: $3 < 5$ なので、そのまま

元の位置	値	図示
2	3	###
6	5	#####

グループ3: $1 > 6$ なので、そのまま

元の位置	値	図示
3	1	#
7	6	#####

グループ4: $2 > 4$ なので、そのまま

元の位置	値	図示
4	2	##
8	4	####

3. 統合して、元の位置に戻します。ただし、**入れ替えたところは、入れ替えた順序で戻します。**

元の位置	値	図示
1	7	#####
2	3	###
3	1	#
4	2	##
5	8	#####
6	5	#####
7	6	#####
8	4	####

これにより、大きな値がだいたい右へ、小さな値がだいたい左へ一気に移動します。

【ステップ2：間隔「2」で整列】

1. 2つ飛ばしのグループに分けます

グループ1

元の位置	値	図示
1	7	#####

元の位置	値	図示
3	1	#
5	8	#####
7	6	#####

グループ2

元の位置	値	図示
2	3	###
4	2	##
6	5	#####
8	4	####

2. 各グループで挿入ソートを行います。

挿入ソートのやり方は以前の章を参照してください。結果だけを示します。

グループ1

元の位置	値	図示
1	1	#
3	6	#####
5	7	#####
7	8	#####

グループ2

元の位置	値	図示
2	2	##
4	3	###
6	4	####
8	5	#####

3. ポジションを戻します

元の位置	値	図示
1	1	#
2	2	##
3	6	#####

元の位置	値	図示
4	3	###
5	7	#####
6	4	####
7	8	#####
8	5	#####

かなり整列された状態に近づきます。

【ステップ3：間隔「1」で整列】

- 最後に普通の挿入ソートを行います。

挿入ソートのやり方はその章を参照してください。結果だけを示します。

元の位置	値	図示
1	1	#
2	2	##
3	3	###
4	4	####
5	5	#####
6	6	#####
7	7	#####
8	8	#####

- 既に「だいたい並んでいる」状態からスタートするため、挿入ソートの強みが最大限に発揮され、一瞬で終わります。

4. シェルソートの特徴（メリット・デメリット）

項目	特徴
考え方	離れた要素同士で比較・交換し、徐々に間隔を狭めていく。
メリット	普通の挿入ソートよりも 格段に速い 。
デメリット	どの「間隔」で進めるかによって速度が変わる（ベストな間隔の決め方は難しい）。
計算量	$(O(n^{1.25})) \sim (O(n^{1.5}))$ 程度（間隔に依存しますが、よりずっと速いです）。

実装

疑似言語による実装

シェルソートは、挿入ソートの「遠く離れた場所にある要素の移動に時間がかかる」という弱点を、最初は間隔を大きく取って大まかに整列させることで克服したアルゴリズムです。

2023年以降の新形式（現行仕様）に基づいた、シェルソートの疑似言語実装です。

```

o shellSort(整数型の配列: data)
  整数型: n, h, i, j, tmp
  n ← dataの要素数
  h ← n ÷ 2  /* 最初の間隔を決める */
  while (h > 0)
    for (i を h から n - 1 まで 1 ずつ増やす)
      tmp ← data[i]
      j ← i
      /* 間隔 h 離れた要素と比較して挿入ソートを行う */
      while (j ≥ h and data[j - h] > tmp)
        data[j] ← data[j - h]
        j ← j - h
      endwhile
      data[j] ← tmp
    endfor
    h ← h ÷ 2  /* 次の間隔を狭める */
  endwhile

```

このコードのポイント：

- **3重のループ構造:** 一番外側で間隔 h を制御し、その中で間隔 h ごとの挿入ソートを繰り返します。
- **効率性:** 最終的に h が1になるときには、データが「だいたい並んでいる」状態になっているため、最後の挿入ソートが非常に高速に終わります。
- **新形式のルール:** `while...endwhile` や `for...endfor` でブロックを閉じ、代入には $\leftarrow$ を使用しています。

シェルソートの「間隔を狭めていく」イメージは掴めましたか？

C言語による実装

先ほどの疑似言語に基づいた、シェルソートのC言語での実装例です。

```

#include <stdio.h>

void shellSort(int arr[], int n) {
    int h, i, j, tmp;
    // 間隔 h を要素数の半分から始め、1になるまで半減させる
    for (h = n / 2; h > 0; h /= 2) {
        // 決まった間隔 h で挿入ソートを繰り返す
        for (i = h; i < n; i++) {
            tmp = arr[i];
            j = i;

```

```
        // 離れた場所にある要素を一気に移動させる,
        while (j >= h && arr[j - h] > tmp) {
            arr[j] = arr[j - h];
            j -= h;
        }
        arr[j] = tmp;
    }
}

int main() {
    int data[] = {8, 3, 1, 2, 7, 5, 6, 4}; // シェルソートの説明で使った例
    int n = 8;

    printf("ソート前: ");
    for (int i = 0; i < n; i++) printf("%d ", data[i]);
    printf("\n");

    shellSort(data, n);

    printf("ソート後: ");
    for (int i = 0; i < n; i++) printf("%d ", data[i]);
    printf("\n");

    return 0;
}
```